

Philips AMD · Advanced Agentic AI Bootcamp

Pre-Read for Participants

A 30-minute primer to arrive Day 1, Hour 1 ready to build — not to learn fundamentals.

2-day in-person bootcamp · Open-source stack · Guardrails in every lab

How to use this This is a light read, not homework. Skim the concepts, open a couple of the links that are new to you, and come with questions. You do not need to install anything — your lab environment is provisioned for you. Everything here is reinforced hands-on across the two days; the goal is shared vocabulary so we can start building in the first hour.

1. What We Already Assume You Know

This is an advanced cohort. We assume you are comfortable with the ideas below. If any are genuinely new, the linked primer is worth ten minutes before Day 1.

- **What an LLM agent is** — a model given a goal, a set of tools, and a loop: reason → act → observe → repeat, until the goal is met.
- **Prompting and tool-calling** — that models can be asked to emit structured calls to functions, and that the surrounding code executes them and feeds results back.
- **Basic Python and Git** — you will write and commit code in every lab. We use uv for environments; no prior uv experience needed.

Primer if you want a refresher: [Anthropic — Building effective agents](#) (concise, framework-neutral).

2. The Stack You'll Build On

The whole environment runs on open-source, self-hosted components — no external LLM APIs. You don't need to set these up, but recognising the names on Day 1 helps.

Component	What it does in the lab	Skim before Day 1
Open-source models	The reasoning and coding models behind your agents. A larger model for reasoning/agent logic, a code-specialised model for generation — all served locally.	Owen models (Hugging Face)
vLLM	The inference server that hosts the models and exposes a standard API your agent tools already know how to call (an industry-standard format — your models stay fully local). This is what lets 20+ of us share the models efficiently.	docs.vllm.ai
LiteLLM	The gateway in front of the models — one endpoint, per-person keys, rate limits, and a full audit log. You'll meet it as both plumbing and a live guardrail.	docs.litellm.ai
FastAPI	The Python web framework your MCP server and any AI service is built on. Async-native and Pydantic-typed — the validation seams your guardrails plug into. See Section 3.5.	fastapi.tiangolo.com
MCP	Model Context Protocol — the open standard for exposing tools/data to agents as authenticated tools.	modelcontextprotocol.io

	You will build a custom MCP server (a FastAPI app) on Day 1.	
Agent frameworks	CrewAI, LangChain, LangGraph and the OpenAI Agents SDK — for orchestration and multi-agent topologies. You'll pick per task; no need to master all four.	LangGraph docs
OpenCode	The terminal-native AI coding assistant you'll use for spec-driven / vibe coding of the case studies — describe intent, it scaffolds, you validate.	opencode.ai/docs

3. Four Concepts Worth Five Minutes Each

3.1 The agent loop and multi-agent topology

Real systems rarely use one agent. You'll design an orchestrator that delegates to domain agents with typed handoffs between them. The design questions — where to split responsibility, how agents pass structured state, what happens when one fails — are the heart of Day 1's architecture session.

Skim: [LangGraph — multi-agent concepts](#)

3.2 MCP — tools as a protocol, not glue code

Instead of hand-wiring each integration, MCP gives agents a standard way to discover and call tools and data sources — think of it as a universal adapter between models and systems. In the lab you'll build a server that exposes real APIs as authenticated tools, with an auth layer, a rate limiter, and an audit log.

Skim: [What is MCP? \(official intro\)](#)

3.3 Guardrails as architecture — the four layers

Guardrails in this programme are not a lecture; they are code you write in every lab. The model to hold in your head is four layers, each a checkpoint in the request path:

- **Layer 1 — Input guardrail (pre-hook):** validates every input before the agent reasons on it.
- **Layer 2 — Tool-call guard (intercept):** wraps every tool call — schema check, call budget, API allowlist.
- **Layer 3 — Output validator (post-hook):** scans model output before it becomes an action or reaches a person (PII, proprietary-code, confidence).
- **Layer 4 — Governance gate (audit):** human approval for consequential actions, plus a full structured audit trail.

Every lab follows the same six-step cycle: **Define** → **Pre-Hook** → **Build** → **Post-Hook** → **Attack & Validate** → **Commit**. You define the agent, wire the pre-hook, build and run, wire the post-hook, then try to break your own hooks before committing.

3.4 The threat landscape you'll be defending against

Day 2 opens with a red-team warm-up — you'll attack each other's Day 1 systems. To make that productive, it helps to arrive knowing the shape of the risks. Two OWASP references are the grounding for the whole security thread: the LLM Top 10 (application-level risks like prompt injection and insecure output handling) and the newer Agentic Top 10 (agent-specific risks like goal hijacking, excessive agency, and memory poisoning).

- [OWASP Top 10 for LLM Applications](#) — prompt injection, output handling, supply chain, excessive agency.
- [OWASP Top 10 for Agentic Applications](#) — the agent-specific risks behind the red-team exercise.

3.5 FastAPI — the foundation under your MCP server

Every MCP server and AI service you build sits on FastAPI, so it's worth knowing why it's the default choice for AI apps — and what a production-ready one looks like.

Why FastAPI for AI apps specifically:

- **Async-native.** AI apps are I/O-bound — they spend most of their time waiting on model inference and tool calls. Async lets one worker handle many concurrent waits, which is exactly the 20-users-sharing-a-model situation.
- **Pydantic-typed.** Request and response models validate automatically at the boundary — which is not incidental here: a request model is your pre-hook, a response model is your post-hook. The framework hands you the guardrail seams.
- **Self-describing.** Auto-generated OpenAPI docs mean every tool your MCP server exposes documents itself.
- **Ecosystem default.** MCP server examples, agent-serving tutorials, and most of the stack assume it.

The 10-point scaffold — what "production-ready" means here. Every AI service in the labs is built to this checklist. You'll recognise it on Day 1; arrive knowing the shape of it. Several points map directly onto the four guardrail layers from 3.3.

#	Scaffold requirement	Maps to
1	Type hints everywhere — the whole service is typed	Correctness
2	Pydantic v2, strict mode — no silent coercion of inputs	Guardrail L1
3	Async I/O — every model/tool call is awaited, never blocking	Concurrency
4	Typed, resilient clients — timeouts, retries on every outbound call	Reliability
5	Ports & adapters — model/provider behind an interface, swappable	Architecture
6	Validated tool calls — every tool input checked against a schema	Guardrail L2
7	Guarded destructive tools — consequential actions gated, not automatic	Guardrail L4
8	Request + response models — both boundaries typed and validated	Guardrail L1 + L3
9	Contract tests in CI — the guardrail suite runs on every commit	S7 CI gate
10	A pure core — business logic isolated from framework and I/O	Testability

Skim: [FastAPI documentation](#)

4. How This Maps to the Two Days

So the pre-read connects directly to what you'll build, here's the shape of the programme. Nothing to prepare — just orientation.

Day 1 — Build the agent stack	Day 2 — Harden, govern, prove
S0 · Calibration + stack setup S1 · Agent architecture for real systems S2 · MCP server — build & deploy S3 · Case Study 1: Documentation Automation S4 · Case Study 2: Content Creation	S5 · Red-team warm-up + threat landscape S6 · All 4 guardrail layers — implementation S7 · Security automation + controlled execution S8 · Case Study 3: Self-Healing AI System S9 · Governance + data protection S10 · Capstone + showcase

5. Come Ready With

- **A laptop** — you'll connect to a provisioned lab environment in the browser/terminal; nothing to install beforehand.
- **One real workflow in mind** — a documentation or process task from your own work. The case studies land harder when you map them onto something you actually own.
- **A skeptic's eye** — the programme's core habit is never trusting AI output blindly. Bring the questions you'd ask before shipping an agent to production.

Optional stretch (only if keen): read one OWASP Agentic risk in full and come with an example of how it might show up in a documentation-automation agent. That's exactly the thinking Day 2 rewards.

Reference Links

Every link below was verified current. Bookmark the ones relevant to your role.

- [Building effective agents \(Anthropic\)](#)
- [Model Context Protocol — home](#)
- [MCP — getting started intro](#)
- [FastAPI documentation](#)
- [vLLM documentation](#)
- [LiteLLM documentation](#)
- [OpenCode documentation](#)
- [Qwen models \(Hugging Face\)](#)
- [LangGraph — multi-agent concepts](#)
- [OWASP Top 10 for LLM Applications](#)
- [OWASP Top 10 for Agentic Applications](#)